

Progetto di Laboratorio di Sistemi Operativi

Traccia scelta - La Videoteca

Roberto Saiello
N86004076

Gabriele Tracanna
N86004048

Luca Piccolo
N86004407

Febbraio 2026

1 Architettura

Il progetto presenta un'architettura *client-server*, dove entrambi i componenti sono scritti in linguaggio C.

- **Server:** per la gestione di richieste provenienti da client concorrenti, l'architettura adotta un approccio **multi-threaded**. Rispetto a una gestione sequenziale, questo paradigma riduce significativamente i **tempi di attesa** dei client e rende il sistema indipendente dalla durata delle singole connessioni. A livello implementativo, dopo aver eseguito le funzioni di inizializzazione (*socket*, *bind* e *listen*), il server entra in un ciclo di ascolto (*while*) in cui invoca la funzione *accept*. Per ogni nuova connessione stabilita, viene generato un thread dedicato tramite *pthread_create* per gestire la comunicazione con il singolo client.
- **Client:** per discriminare e gestire efficacemente i messaggi sincroni e asincroni, il client impiega due thread con ruoli distinti. Il *listener_thread* intercetta tutto il traffico in ingresso: se rileva un messaggio asincrono, lo elabora in autonomia; in caso contrario, cede il controllo del canale al *main_thread*, che si occupa di ricevere i messaggi di protocollo del server, generati in risposta ai comandi dell'utente.
- **Comunicazione:** lo scambio di messaggi in rete tra client e server avviene tramite socket TCP/IP. Questa scelta garantisce un'elevata affidabilità: i pacchetti vengono consegnati senza perdita di dati e mantenendo rigorosamente l'ordine di invio. La comunicazione si basa su un protocollo testuale a righe (ogni messaggio corrisponde a una singola riga di testo). Questo approccio è stato adottato per la sua semplicità, in quanto agevola sia la lettura dei comandi in linguaggio naturale, sia le operazioni di parsing per la codifica e decodifica in fase di I/O.

2 Server

2.1 Gestione delle connessioni

Come accennato in precedenza, il server gestisce la concorrenza dei client adottando un modello **multi-threaded**. Nello specifico, per ogni nuova connessione stabilita viene generato un thread dedicato (tramite la primitiva *pthread_create*), incaricato di servire le richieste di quel client in totale indipendenza.

Per la gestione del **ciclo di vita** di tali thread viene impiegata la funzione *pthread_detach*. Questa scelta permette a ciascun thread di rilasciare autonomamente le proprie risorse al termine dell'esecuzione, sollevando il processo principale dall'obbligo di attenderne la terminazione. Così facendo, si evitano attese bloccanti e si previene efficacemente il rischio di memory leak e la formazione di zombie thread.

La scelta di adottare un'architettura multi-thread, al posto di creare un processo per ogni client, nasce principalmente dalla necessità di gestire in modo semplice e diretto le **risorse condivise**. Infatti, ciò permette di mantenere centralizzate le strutture dati delle risorse (come film e prenotazioni) in memoria globale, eliminando quindi la complessità di gestire ed implementare meccanismi per la comunicazione tra processi. Inoltre, l'utilizzo dei thread garantisce anche **vantaggi prestazionali**, in quanto permette di evitare al sistema operativo di dover copiare l'intero spazio di indirizzamento per ogni nuova connessione stabilita.

2.2 Strutture dei dati

Ogni risorsa della business logic è rappresentata da un'apposita struttura "risorsa.t" presente di seguito:

1. **user.t**

- ID univoco.
- Email univoca.
- Password

2. **film.t**

- ID univoco.
- Titolo.
- Numero di copie disponibili.
- Numero di copie noleggiate.

3. **reservation.t**

- ID univoco.

- Data di noleggio.
- Data di scadenza del noleggio.
- Data di restituzione del film.
- ID del film noleggiato.
- ID dell'utente.

4. **connection_t**

- ID dell'utente.
- TID (*thread id*) del server assegnato al client connesso.
- PID (*process id*) del client connesso.
- FD (*file descriptor*) della socket aperta dal client connesso.

Tali risorse saranno presenti, in ordine, nelle strutture globali **user_list_t**, **film_list_t**, **reservation_list_t** e **connection_list_t**. Infatti, le strutture globali, identificate dalla terminazione "_list_t", presenteranno un array per collezionare le singole risorse per tipologia, ed un mutex per regolarne l'utilizzo concorrente da parte dei vari thread.

2.3 Gestione della concorrenza

Data l'adozione di un'architettura **multi-threaded**, l'accesso concorrente alle risorse condivise rende indispensabile l'impiego di meccanismi di sincronizzazione, al fine di prevenire race condition e garantire l'integrità dei dati.

Anziché definire i mutex per singole sezioni critiche all'interno del codice, si è scelto di incapsulare ciascun mutex direttamente nella specifica struttura dati di cui deve proteggere lo stato.

Questa soluzione garantisce un'elevata granularità del controllo: thread diversi possono operare simultaneamente su collezioni di dati distinte senza bloccarsi a vicenda. Nello specifico, sono stati introdotti i seguenti mutex:

- **users_mutex:** contenuto in *user_list_t*, protegge l'accesso concorrente all'array degli utenti ed ai relativi contatori. Garantisce l'integrità dei dati durante le fasi di registrazione, autenticazione o ricerca di un utente.
- **films_mutex:** contenuto in *film_list_t*, protegge l'accesso concorrente all'array dei film ed ai relativi contatori. Previene conflitti quando più client tentano contemporaneamente il catalogo, noleggiare oppure restituire film.
- **reservations_mutex:** contenuto in *reservation_list_t*, protegge l'accesso concorrente all'array delle prenotazioni ed ai relativi contatori. Sincronizza le modifiche sullo stato di un noleggio, in seguito alla restituzione di un film noleggiato.

- **connection_mutex:** contenuto in *connection_list_t*, protegge l'accesso concorrente all'array delle connessioni ed ai relativi contatori. Gestisce l'inserimento dei dati relativi alle connessioni dei client.
- **shopkeeper_max_rented_films_mutex:** protegge l'accesso alla variabile globale "shopkeeper_max_rented_films_per_user", assicurando che la lettura e l'aggiornamento del limite di film noleggiabili dagli utenti avvenga in modo atomico.

2.4 Invio messaggi asincroni

Per consentire al gestore della videoteca di inviare **notifiche globali** a tutti gli utenti, ovvero un promemoria per la restituzione dei film, il server implementa un meccanismo di **comunicazione asincrona** basato sul **broadcasting**.

Una volta ricevuto l'apposito messaggio di protocollo dal client con privilegi di negoziante, il server innesca una procedura **indipendente** dal modello richiesta-risposta utilizzato, invece, per i messaggi sincroni: **itera** sulla struttura dati delle **connessioni attive**, accede al file descriptor delle socket di ciascun client e scrive direttamente il messaggio di notifica.

Tali messaggi verranno successivamente intercettati lato client da un thread dedicato **listener_thread** ed elaborati in background, garantendo che le notifiche raggiungano tutti gli utenti senza bloccare i terminali corrispondenti.

2.5 Persistenza Dati

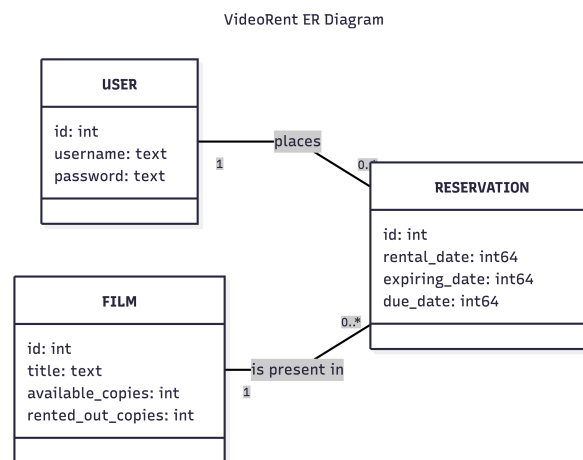


Figure 1: Diagramma ER

La persistenza dei dati all'interno del server è affidata a **SQLite3**, scelto per

la sua natura minimale e la **facile integrazione** con il linguaggio C.

La gestione delle informazioni si basa su un principio di **doppia rappresentazione**: una **relazionale**, basata sulla persistenza dei record nel database, ed una **in memoria**, dove i dati degli stessi record sono contenuti in specifiche *struct* globali.

Al fine di garantire l'integrità dei dati, le due rappresentazioni vengono mantenute **sincronizzate** tra loro durante ogni singola operazione CRUD.

Infine, per assicurare la **continuità operativa**, allo startup del server apposite routine si occupano di interrogare il database per recuperare i record salvati e **popolare dinamicamente** le strutture in memoria, ripristinando lo stato globale dell'applicazione.

Le funzioni dedite all'interazione con il database sono:

- Famiglia **_table_init**: creano ed inizializzano le tabelle del database al primo avvio del server, qualora non siano già presenti.
- Famiglia **_list_sync**: popolano dinamicamente le strutture dati in memoria recuperando i record dal database durante la fase di startup.
- Famiglia **_insert**: gestiscono l'aggiunta di nuovi record all'interno del database tramite apposite query INSERT
- **Operazioni di aggiornamento**: riflettono sul database le modifiche di stato derivanti dalla logica applicativa (come il noleggio o la restituzione di un film), garantendo la costante coerenza dei dati.

2.5.1 Schema logico

- USER(id, username, password)
 - USER.username UNIQUE
- FILM(id, title, available_copies, rented_out_copies)
- RESERVATION(id, rental_date, expiring_date, due_date, user_id, film_id)
 - RESERVATION.user_id → USER.id
 - RESERVATION.film_id → FILM.id

3 Client

3.1 Gestione della concorrenza interna

Al fine di garantire la capacità dell'applicazione di ricevere le notifiche inviate da un negoziante in modo asincrono e, quindi, prevenire problemi di *race condition* in lettura sulla singola socket tra queste ed i normali messaggi sincroni, l'architettura interna del client segue il pattern "**Listener Thread con passaggio di consegne**".

Questa soluzione prevede la creazione di un **thread secondario**, chiamato `listener_thread`, al quale è delegata l'esclusiva responsabilità di leggere costantemente tutti i messaggi di protocollo dalla socket ed, eventualmente, passarli al `main_thread`.

Per la gestione della concorrenza, il `listener_thread` ed il `main_thread` utilizzano la seguente struttura:

- `threads_sync_t`
 - `sync_mutex`
 - `wake_main_thread_cv`
 - `wake_listener_thread_cv`
 - `data_for_main_thread_ready`
 - `listener_suspended`
 - `server_response`

In particolare, `wake_main_thread_cv` e `wake_listener_thread_cv` sono variabili di condizione che servono per sbloccare corrispettivamente il `main_thread` ed il `listener_thread` quando sono in attesa, `data_for_main_thread_ready` serve per segnalare al `main_thread` che è presente un messaggio sincrono da "consumare", `listener_suspended` è utilizzata dal `listener_thread` stesso per entrare in attesa e passare il controllo della socket al `main_thread`, ed infine `server_response` rappresenta il buffer dove il `listener_thread`, prima di mettersi in attesa, copia il messaggio di protocollo sincrono per permettere successivamente al `main_thread`, quando risvegliato, di leggerlo.

I messaggi vengono smistati secondo la seguente logica:

- Se letto il messaggio "SHOW_EXPIRED_FILMS_NOTIFICATION" **asincrono**, ovvero la notifica dei film scaduti, il thread aggiorna la variabile globale "*film_reminder*" senza bloccare il flusso di esecuzione principale, in modo tale da non alterare il flusso del menù utente. Si noti che **non viene effettuato** alcun passaggio di consegne verso il `main_thread`
- Se letti tutti gli altri messaggi, quindi **sincroni**, il `listener_thread` innesca invece la procedura di **passaggio di consegne**. Attraverso l'uso coordinato della

struttura globale `threads_sync.t` per la sincronizzazione, il `listener_thread` salva il messaggio in una struttura dati condivisa, notifica il `main_thread` e sospende temporaneamente la propria lettura della socket. Questo approccio cede il **controllo esclusivo** della socket al thread principale, garantendogli la possibilità di leggere e scrivere in totale sicurezza ulteriori dati aggiuntivi (come le strutture dati dei film o l'ID utente), per poi riattivare il listener ad operazione conclusa, per permettere la lettura dei prossimi messaggi di protocollo, sincroni oppure asincroni.

3.2 Menù

All'avvio del client, il sistema presenta all'utente tre opzioni: **registrazione** di un nuovo account, **accesso** tramite credenziali esistenti (login) oppure **chiusura** dell'applicazione. A seguito della registrazione, è richiesto il login per accedere all'interfaccia principale.

Il **menu principale** è strutturato in tre sezioni distinte: il **catalogo** dei titoli, lo stato del **carrello** e le **azioni disponibili**.

Il catalogo dei film riporta i **dati essenziali** per il noleggio: l'ID univoco, il titolo, le copie attualmente disponibili e quelle già noleggiate.

Invece, la sezione dedicata al **carrello** mostra i titoli selezionati per il noleggio, in alternativa, segnala se risulta vuoto.

Le **funzioni** a disposizione dell'utente comprendono: l'inserimento di un film nel carrello, la sua modifica, la finalizzazione dell'ordine (checkout), la visualizzazione dei film attualmente noleggiati e la relativa restituzione.

Qualora il negoziante invii una **notifica** di scadenza, il sistema mostrerà in evidenza la lista dei film con noleggio scaduto.

L'interfaccia riservata al **negoziante** mantiene la medesima struttura, differenziandosi unicamente per le specifiche funzionalità di gestione a disposizione.

3.3 Interazione utente

L'interazione avviene tramite una **TUI** (Terminal User Interface) dinamica e contestuale: ogni selezione mostra un **sotto-menù** che presenta esclusivamente le informazioni pertinenti all'operazione in corso.

Il sistema quindi, ad ogni cambio di schermata, garantisce l'**aggiornamento** dei dati. Inoltre, tale meccanismo di refresh, permette la visualizzazione immediata delle **notifiche** di restituzione non appena l'utente cambia sezione.

Le funzionalità sono presentate tramite **elenchi numerati** (es. 1 - **Inserisci ID**, 2 - **Modifica carrello**) indipendenti dal ruolo dell'utente. L'input avviene tramite l'inserimento dell'**indice numerico** corrispondente, mentre le procedure che richiedono conferma vengono gestite tramite l'acquisizione del carat-

tere iniziale della scelta (s/n).

Per la **formattazione** delle tabelle è stato adottato il meccanismo del **Minimum Field Width** (MFW). Questa tecnica permette di allineare i dati definendo una larghezza fissa per ogni colonna: lo spazio totale occupato da ogni campo comprende i caratteri effettivamente stampati e i necessari spazi di **riempimento** (padding) per garantire l'incolonnamento costante.

4 Protocollo di comunicazione

I messaggi di protocollo testuali, sempre immagazzinati in appositi **buffer di dimensione fissa** sia nel client che nel server, sono stati scelti principalmente per **prevenire** in modo sicuro **frammentazioni** o **corruzioni** dei dati durante le fasi di lettura e scrittura sui socket TCP/IP.

In secondo luogo, questo approccio offre l'ulteriore vantaggio di semplificare il **riconoscimento dei comandi**, rendendo la logica del codice immediatamente leggibile e facilitandone le operazioni di correzione.

Per avviare i vari casi d'uso, vengono inviati sulle rispettive socket i seguenti messaggi di protocollo:

- **REGISTER**: il client invia il comando seguito dallo username e dalla password dell'account da registrare. Il server, dopo aver verificato che non esista già un account con lo stesso username, esegue la registrazione.
- **LOGIN**: il client invia il comando e le proprie credenziali. Il server verifica l'esistenza dell'account e la correttezza della password; in caso positivo, esegue l'autenticazione, scrive sulla socket il messaggio dal prefisso **SUCCESS** seguito dall'ID dell'utente e aggiorna lo stato della connessione attiva. Il client, letto il messaggio, memorizza l'ID utente in un'apposita variabile globale.
- **GET_FILMS**: letto il comando, il server risponde scrivendo il messaggio dal prefisso **SUCCESS**. Successivamente recupera l'intero catalogo, scrive il numero totale di film presenti e, tramite un ciclo, scrive i dati di ciascun film.
- **RENT_FILM**: il client scrive il comando, l'ID dell'utente autenticato e l'ID del film da noleggiare. Il server elabora il noleggio previa verifica di tre condizioni: disponibilità di copie (disponibili e noleggate) sufficienti, mancato superamento del limite massimo di film noleggiabili e assenza di un noleggio attivo (non ancora restituito) per lo stesso film da parte dell'utente.
- **RETURN_RENTED_FILM**: il client scrive il comando, l'ID utente e l'ID del film da restituire. Il server, dopo aver verificato la coerenza del numero di copie (disponibili e noleggate), esegue la restituzione.
- **GET_USER_RENTED_FILMS**: il client scrive il comando e l'ID utente. Il server verifica che l'utente sia il negoziante; in caso affermativo, scrive il messaggio dal prefisso **SUCCESS**, recupera i film noleggiati, scrive il numero totale e scrive iterativamente i dati di ciascuno.

- **GET_MAX_RENTED_FILMS:** a seguito della richiesta del client, il server scrive il messaggio dal prefisso **SUCCESS** seguito dal limite massimo di film noleggiabili impostato dal negoziante. Infine, sulla base del limite appena recuperato, il client aggiorna anche la dimensione del carrello.
- **GET_USER_EXPIRED_FILMS_NO_DUE_DATE:** il client scrive il comando e l'ID utente. Il server risponde con il messaggio dal prefisso **SUCCESS**, individua i film con noleggio scaduto per quell'utente, ne scrive la quantità e, tramite un ciclo, scrive l'ID e il titolo di ciascun film.
- **SHOPKEEPER_GET_ALL_RESERVATIONS:** il client scrive il comando e l'ID utente. Il server, accertati i privilegi di negoziante, scrive il messaggio dal prefisso **SUCCESS**, scrive il numero totale delle prenotazioni attive e ne scrive i dettagli iterando su di esse.
- **SHOPKEEPER_CHANGE_MAX_RENTED_FILMS:** il client scrive il comando, l'ID utente e il nuovo limite massimo di noleggi. Il server verifica che l'utente sia un negoziante e che attualmente nessun utente superi il nuovo limite stabilito; in caso positivo, aggiorna la variabile globale e scrive il messaggio con prefisso **SUCCESS**.
- **SHOPKEEPER_NOTIFY_EXPIRED_FILMS:** il client scrive il comando e l'ID utente. Il server, verificato che l'utente sia un negoziante, scrive in broadcast il messaggio **SHOW_EXPIRED_FILMS_NOTIFICATION** sulle socket di tutti i client connessi, per poi confermare l'operazione scrivendo il messaggio dal prefisso **SUCCESS** al client richiedente.
- **SHOW_EXPIRED_FILMS_NOTIFICATION:** scritto dal server in modo asincrono, questo messaggio viene letto dal client, il quale imposta una variabile globale per mostrare la notifica dei noleggi scaduti alla successiva interazione utile da parte dell'utente.

Infine, il server comunica l'esito di ogni operazione antepoendo alla risposta il prefisso **SUCCESS** o **FAILED**. Il client interpreta questi messaggi per mostrare a schermo il corretto testo informativo per l'utente.

4.1 Setup con Docker e Docker Compose

Il progetto adotta Docker per garantire un ambiente di esecuzione isolato e riproducibile, e Docker Compose per orchestrare l'intera architettura multi-servizio attraverso un'unica configurazione centralizzata.

4.1.1 Dockerfile Multi-Stage

Sia per il client che per il server è stata utilizzata l'immagine Ubuntu 24.04, separando la fase di compilazione da quella di esecuzione:

- **Stage di Build:** In questa fase vengono installate le dipendenze necessarie per la compilazione e, nel caso del server, le librerie per la gestione del database `sqlite3`. I sorgenti C vengono compilati includendo le dipendenze per generare l'eseguibile.

- **Stage di Runtime:** L'eseguibile compilato viene copiato in una nuova immagine "scratch" pulita, insieme al file del database di default e alle sole librerie necessarie all'esecuzione, ovvero "libsqlite3-0".

I vantaggi offerti dalla separazione compilazione-esecuzione sono i seguenti:

1. **Ottimizzazione delle dimensioni:** L'immagine finale risulta estremamente più leggera, poiché non include il codice sorgente, gli header file, il compilatore GCC o strumenti come Make.
2. **Pulizia:** Il file system del container finale contiene esclusivamente il binario compilato e i file strettamente necessari al suo funzionamento, non il codice sorgente.

4.1.2 Orchestrazione con Docker Compose

Il server e il client sono definiti come servizi distinti all'interno del file "compose.yaml", collegati tra loro da una rete virtuale interna dedicata "videorent_network".

- **Servizio Server:** Viene avviato esponendo la porta 5200 verso l'host, consentendo anche connessioni esterne. Un aspetto fondamentale di questo servizio è l'utilizzo del volume `videorent_database` montato sulla directory di lavoro: questo garantisce la persistenza dello stesso database SQLite, evitando la perdita delle informazioni in caso di spegnimento o riavvio del container.
- **Servizio Client:** Il client dipende logicamente dal server. Per gestire correttamente le tempistiche di avvio, esegue un comando di ritardo "sleep 2" prima di avviare l'eseguibile, assicurandosi che il server sia già in ascolto. La connessione al server è resa configurabile tramite le variabili d'ambiente `SERVER_HOST` e `SERVER_PORT`. Infine, per permettere all'utente di interagire con il menù testuale, i canali standard di input e output sono mantenuti aperti utilizzando "stdin_open: true" ed "tty: true".